Báo cáo bài tập ngày 16/6/2026

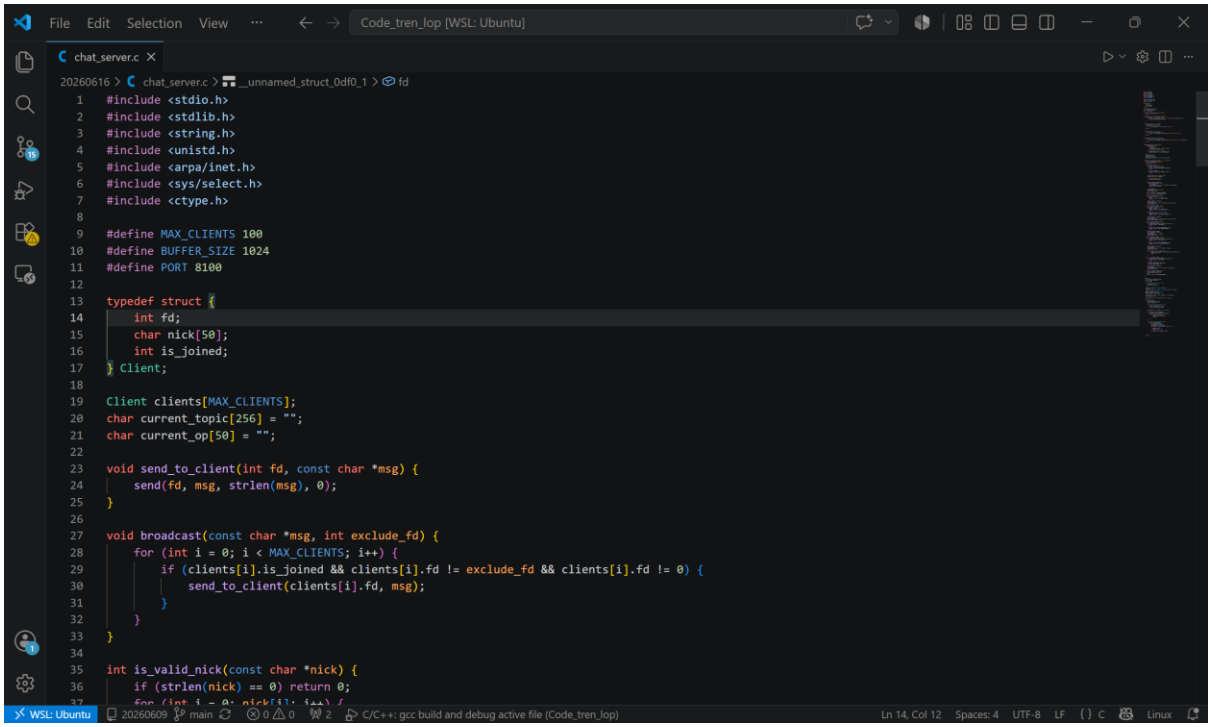
Họ và tên:

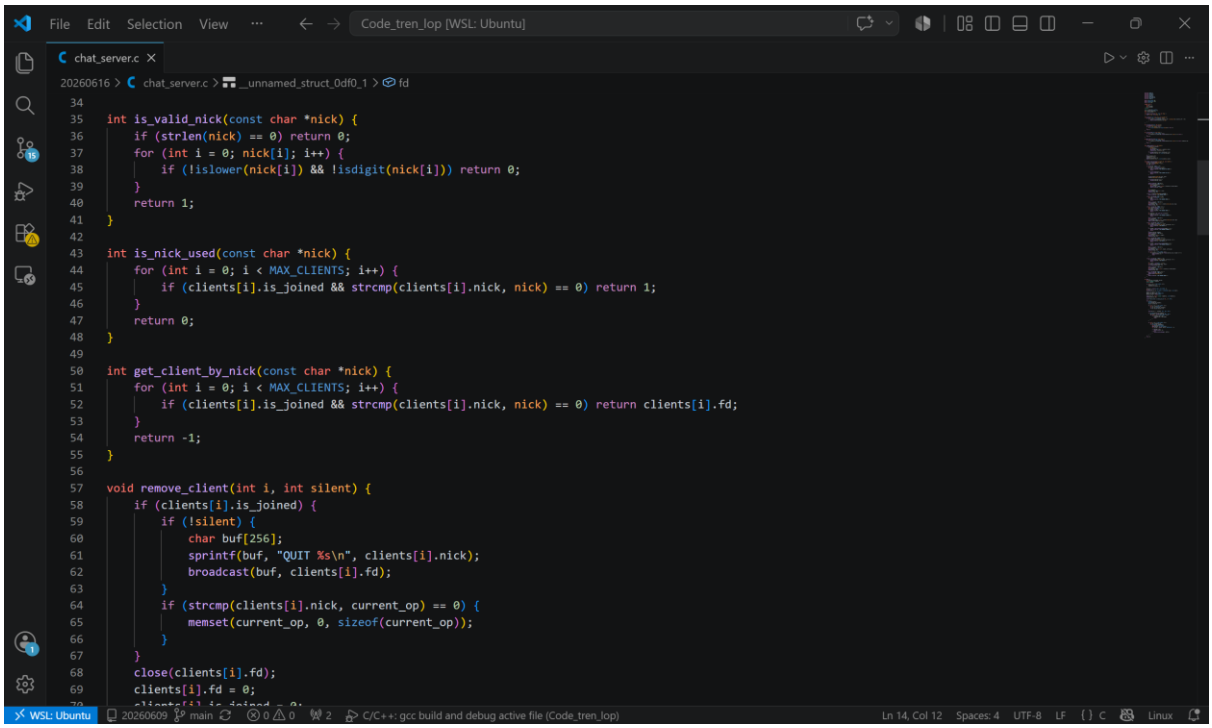
Nguyễn Kim Tùng Quân – 20235407 – Chat_server

Trần Anh Phú – 20235397 – Chat_client

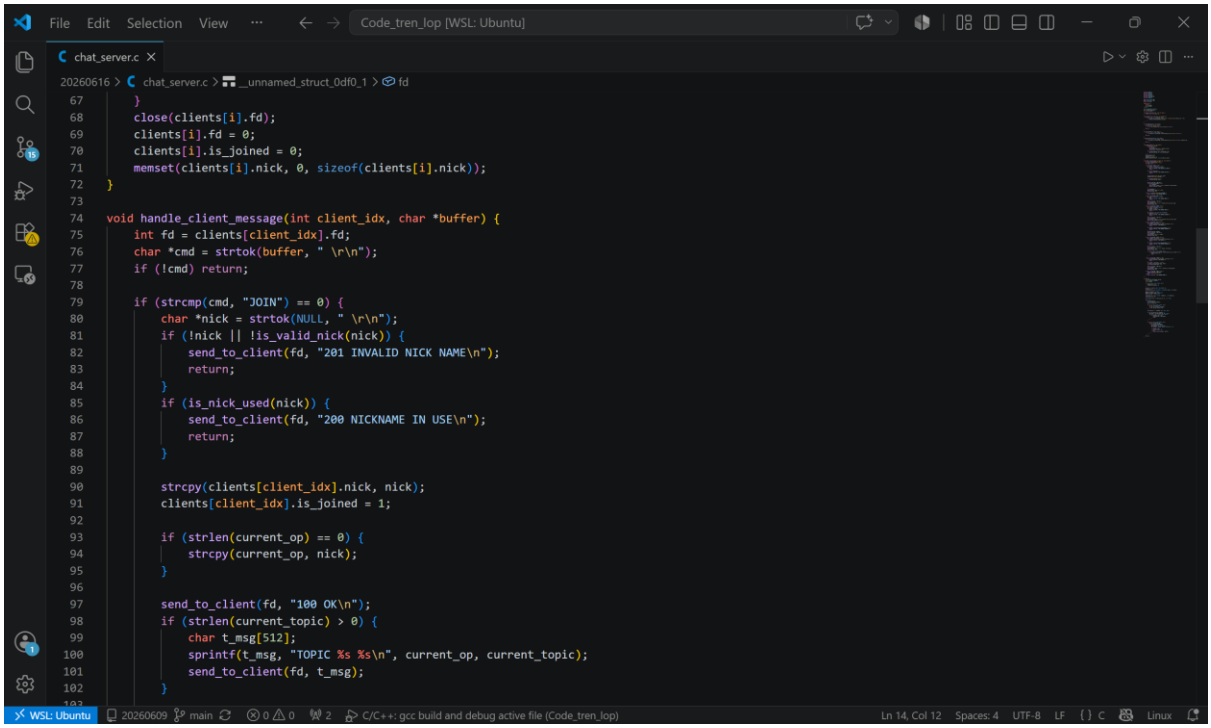
Bài tập: Sinh viên lập trình ứng dụng Chat Server / Client theo giao thức được mô tả trong slide.
Mỗi nhóm 2 sinh viên:
+ Một người lập trình ứng dụng chat server
+ Một người lập trình ứng dụng chat client (sử dụng giao diện tùy ý)
* Đối với ứng dụng server
+ Tải và chạy file kiểm thử tại địa chỉ http://lebavui.io.vn/chat_server_test
+ Cấp quyền thực thi cho file kiểm thử
+ Chạy file kiểm thử và nộp ảnh chụp kết quả

Code Server:

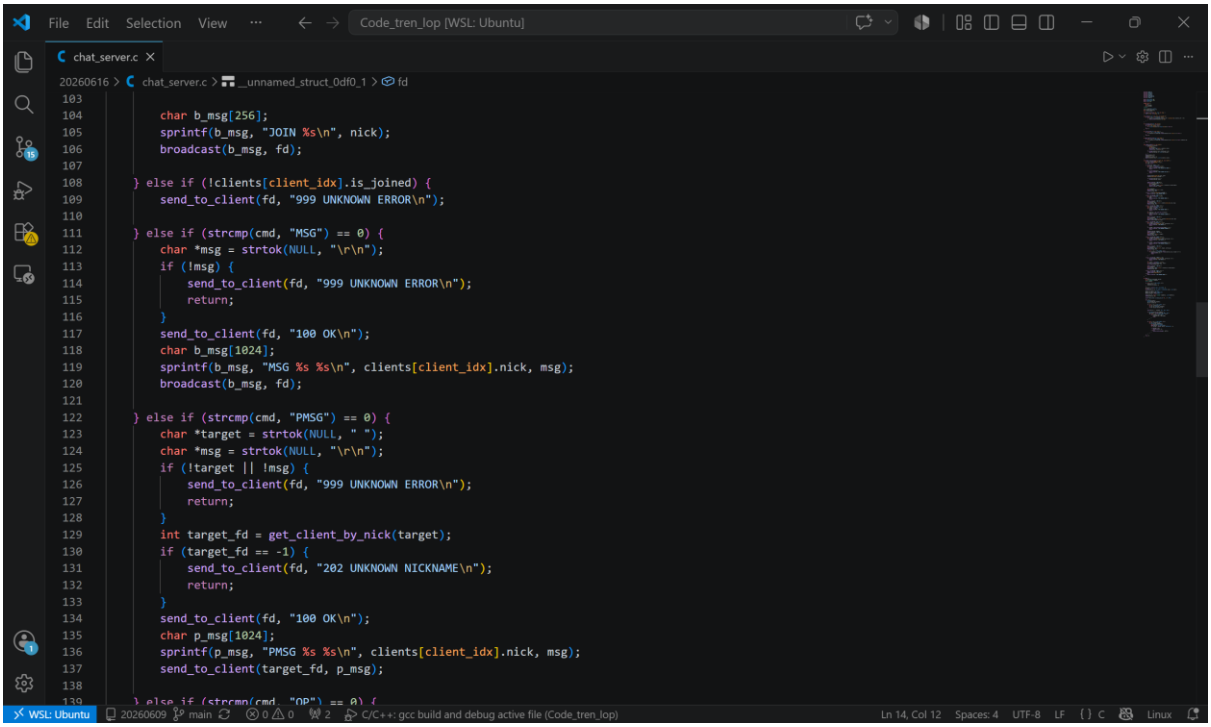




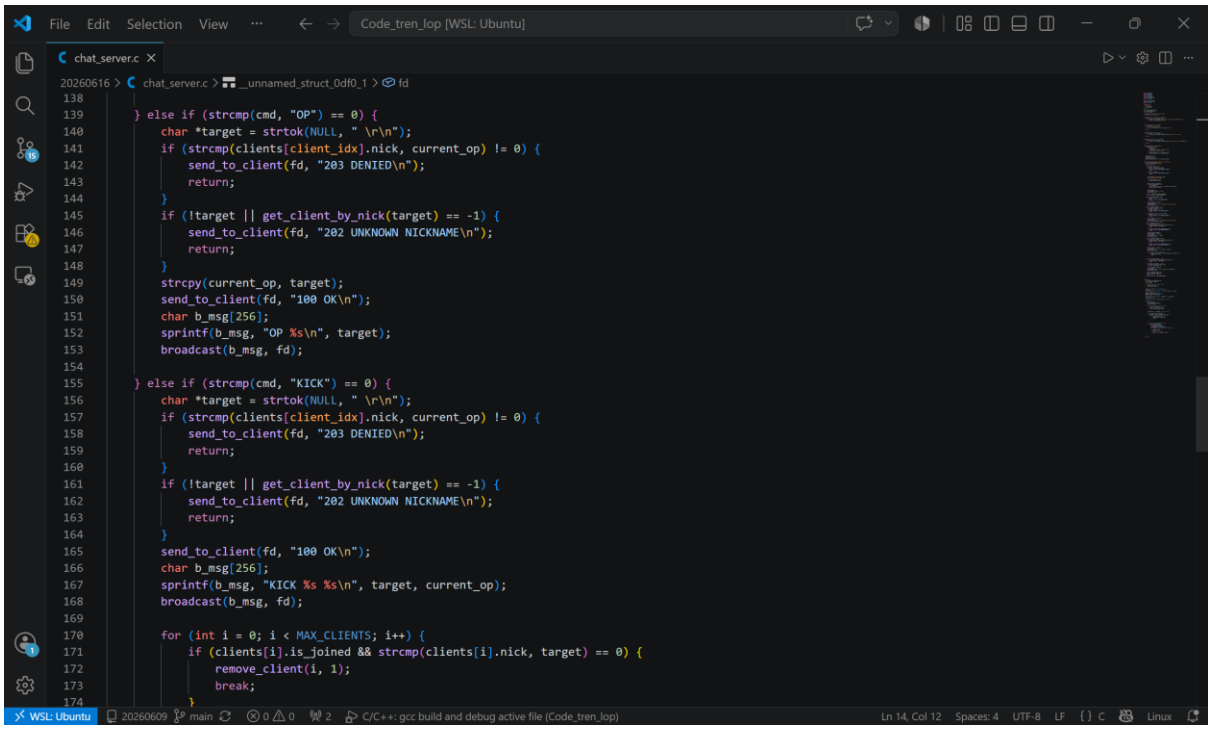
```c
int is_valid_nick(const char *nick) {
    if (strlen(nick) == 0) return 0;
    for (int i = 0; nick[i]; i++) {
        if (!islower(nick[i]) && !isdigit(nick[i])) return 0;
    }
    return 1;
}

int is_nick_used(const char *nick) {
    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (clients[i].is_joined && strcmp(clients[i].nick, nick) == 0) return 1;
    }
    return 0;
}

int get_client_by_nick(const char *nick) {
    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (clients[i].is_joined && strcmp(clients[i].nick, nick) == 0) return clients[i].fd;
    }
    return -1;
}

void remove_client(int i, int silent) {
    if (clients[i].is_joined) {
        if (!silent) {
            char buf[256];
            sprintf(buf, "QUIT %s\n", clients[i].nick);
            broadcast(buf, clients[i].fd);
        }
        if (strcmp(clients[i].nick, current_op) == 0) {
            memset(current_op, 0, sizeof(current_op));
        }
    }
    close(clients[i].fd);
    clients[i].fd = 0;
```



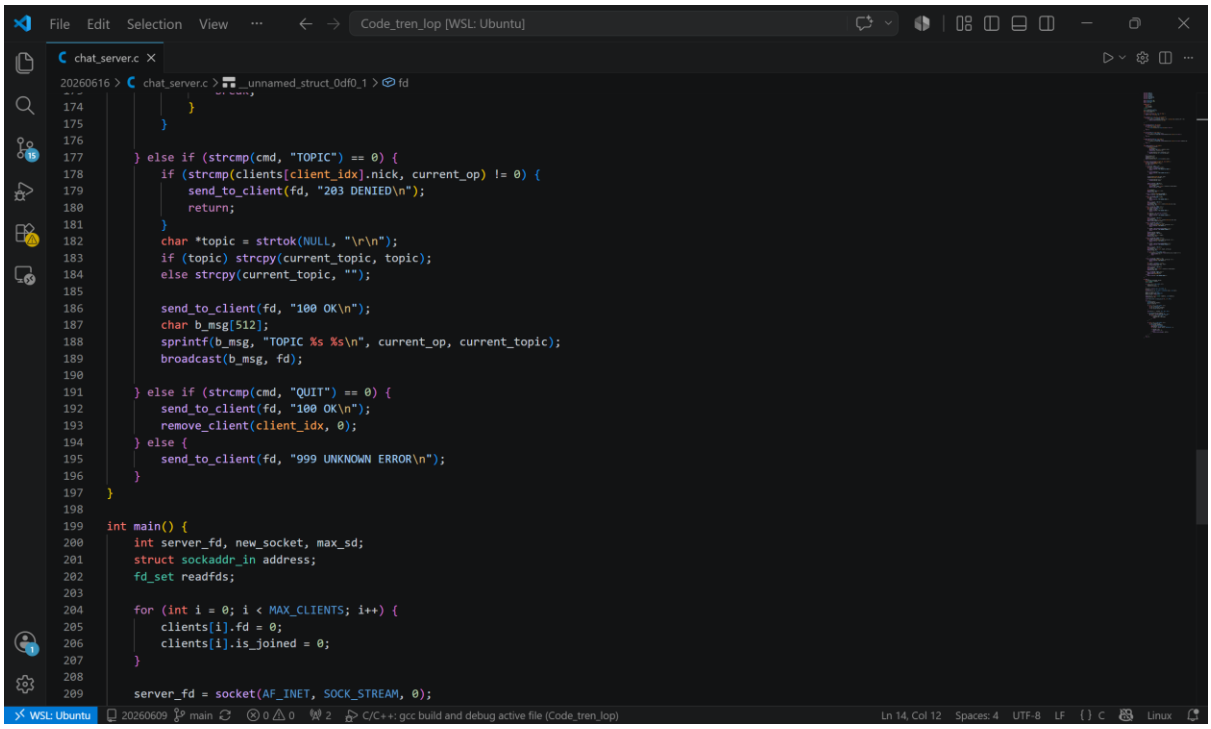
```c
    }
    close(clients[i].fd);
    clients[i].fd = 0;
    clients[i].is_joined = 0;
    memset(clients[i].nick, 0, sizeof(clients[i].nick));
}

void handle_client_message(int client_idx, char *buffer) {
    int fd = clients[client_idx].fd;
    char *cmd = strtok(buffer, " \r\n");
    if (!cmd) return;

    if (strcmp(cmd, "JOIN") == 0) {
        char *nick = strtok(NULL, " \r\n");
        if (!nick || !is_valid_nick(nick)) {
            send_to_client(fd, "201 INVALID NICK NAME\n");
            return;
        }
        if (is_nick_used(nick)) {
            send_to_client(fd, "200 NICKNAME IN USE\n");
            return;
        }

        strcpy(clients[client_idx].nick, nick);
        clients[client_idx].is_joined = 1;

        if (strlen(current_op) == 0) {
            strcpy(current_op, nick);
        }

        send_to_client(fd, "100 OK\n");
        if (strlen(current_topic) > 0) {
            char t_msg[512];
            sprintf(t_msg, "TOPIC %s %s\n", current_op, current_topic);
            send_to_client(fd, t_msg);
        }
```



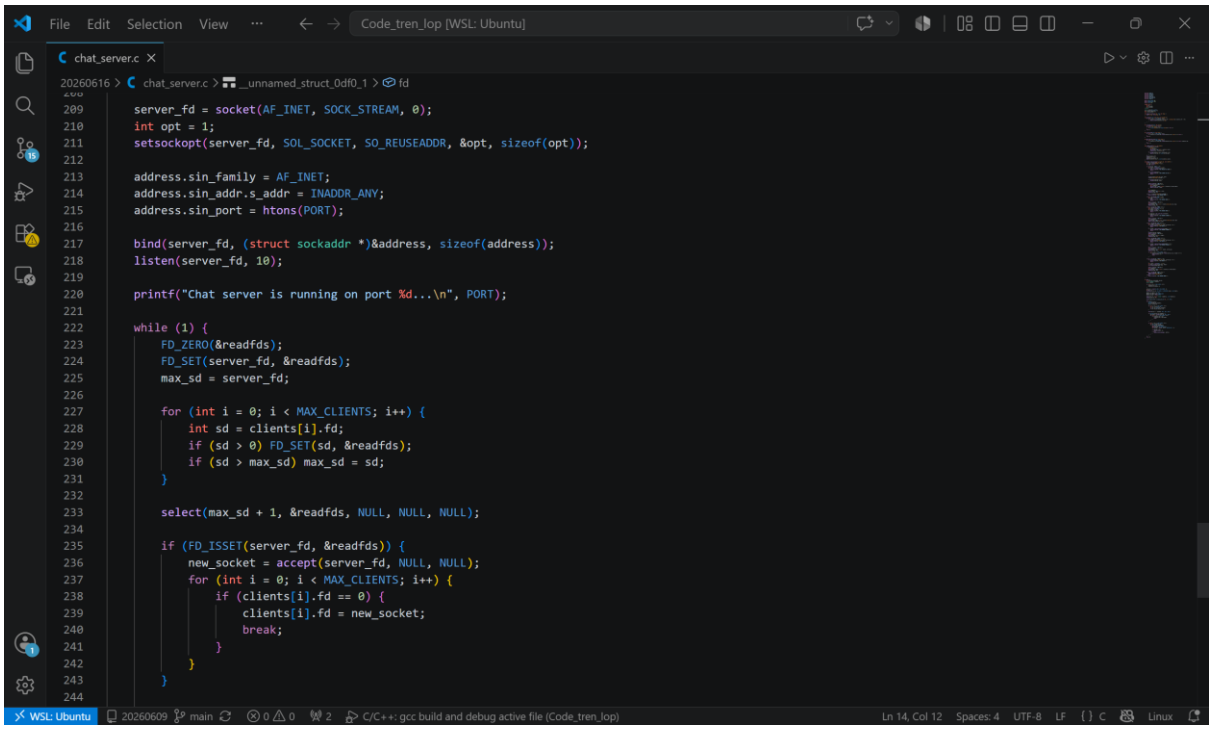
```
103
104         char b_msg[256];
105         sprintf(b_msg, "JOIN %s\n", nick);
106         broadcast(b_msg, fd);
107
108     } else if (!clients[client_idx].is_joined) {
109         send_to_client(fd, "999 UNKNOWN ERROR\n");
110
111     } else if (strcmp(cmd, "MSG") == 0) {
112         char *msg = strtok(NULL, "\r\n");
113         if (!msg) {
114             send_to_client(fd, "999 UNKNOWN ERROR\n");
115             return;
116         }
117         send_to_client(fd, "100 OK\n");
118         char b_msg[1024];
119         sprintf(b_msg, "MSG %s %s\n", clients[client_idx].nick, msg);
120         broadcast(b_msg, fd);
121
122     } else if (strcmp(cmd, "PMSG") == 0) {
123         char *target = strtok(NULL, " ");
124         char *msg = strtok(NULL, "\r\n");
125         if (!target || !msg) {
126             send_to_client(fd, "999 UNKNOWN ERROR\n");
127             return;
128         }
129         int target_fd = get_client_by_nick(target);
130         if (target_fd == -1) {
131             send_to_client(fd, "202 UNKNOWN NICKNAME\n");
132             return;
133         }
134         send_to_client(fd, "100 OK\n");
135         char p_msg[1024];
136         sprintf(p_msg, "PMSG %s %s\n", clients[client_idx].nick, msg);
137         send_to_client(target_fd, p_msg);
138
139     } else if (strcmp(cmd, "OP") == 0) {
```



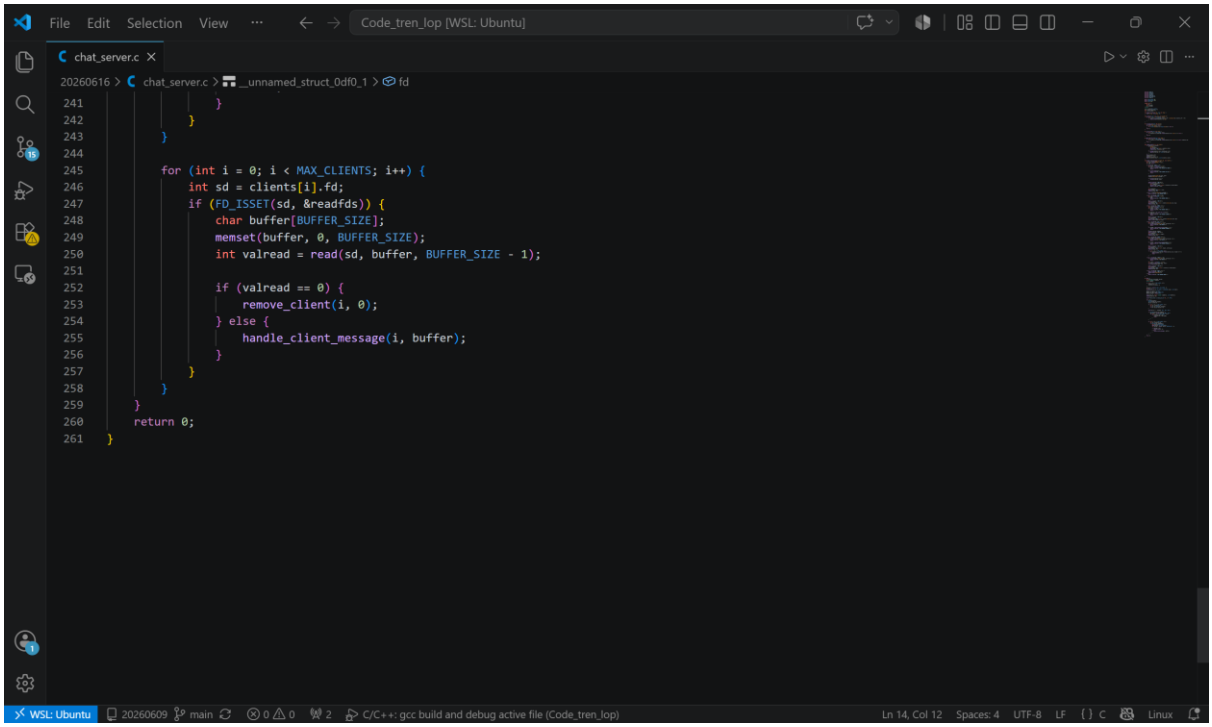
```
138
139     } else if (strcmp(cmd, "OP") == 0) {
140         char *target = strtok(NULL, " \r\n");
141         if (strcmp(clients[client_idx].nick, current_op) != 0) {
142             send_to_client(fd, "203 DENIED\n");
143             return;
144         }
145         if (!target || get_client_by_nick(target) == -1) {
146             send_to_client(fd, "202 UNKNOWN NICKNAME\n");
147             return;
148         }
149         strcpy(current_op, target);
150         send_to_client(fd, "100 OK\n");
151         char b_msg[256];
152         sprintf(b_msg, "OP %s\n", target);
153         broadcast(b_msg, fd);
154
155     } else if (strcmp(cmd, "KICK") == 0) {
156         char *target = strtok(NULL, " \r\n");
157         if (strcmp(clients[client_idx].nick, current_op) != 0) {
158             send_to_client(fd, "203 DENIED\n");
159             return;
160         }
161         if (!target || get_client_by_nick(target) == -1) {
162             send_to_client(fd, "202 UNKNOWN NICKNAME\n");
163             return;
164         }
165         send_to_client(fd, "100 OK\n");
166         char b_msg[256];
167         sprintf(b_msg, "KICK %s %s\n", target, current_op);
168         broadcast(b_msg, fd);
169
170         for (int i = 0; i < MAX_CLIENTS; i++) {
171             if (clients[i].is_joined && strcmp(clients[i].nick, target) == 0) {
172                 remove_client(i, 1);
173                 break;
174             }
```



```
174             }
175         }
176
177     } else if (strcmp(cmd, "TOPIC") == 0) {
178         if (strcmp(clients[client_idx].nick, current_op) != 0) {
179             send_to_client(fd, "203 DENIED\n");
180             return;
181         }
182         char *topic = strtok(NULL, "\r\n");
183         if (topic) strcpy(current_topic, topic);
184         else strcpy(current_topic, "");
185
186         send_to_client(fd, "100 OK\n");
187         char b_msg[512];
188         sprintf(b_msg, "TOPIC %s %s\n", current_op, current_topic);
189         broadcast(b_msg, fd);
190
191     } else if (strcmp(cmd, "QUIT") == 0) {
192         send_to_client(fd, "100 OK\n");
193         remove_client(client_idx, 0);
194     } else {
195         send_to_client(fd, "999 UNKNOWN ERROR\n");
196     }
197 }
198
199 int main() {
200     int server_fd, new_socket, max_sd;
201     struct sockaddr_in address;
202     fd_set readfds;
203
204     for (int i = 0; i < MAX_CLIENTS; i++) {
205         clients[i].fd = 0;
206         clients[i].is_joined = 0;
207     }
208
209     server_fd = socket(AF_INET, SOCK_STREAM, 0);
```

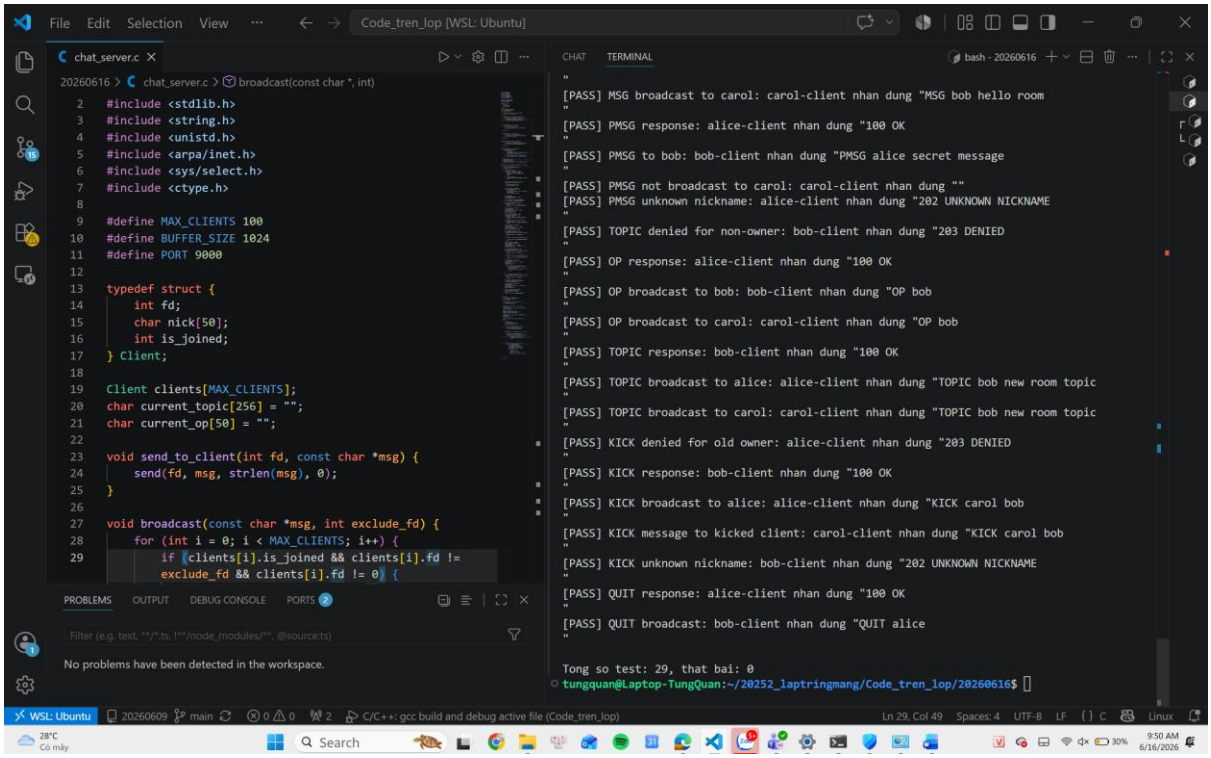


```
209     server_fd = socket(AF_INET, SOCK_STREAM, 0);
210     int opt = 1;
211     setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
212
213     address.sin_family = AF_INET;
214     address.sin_addr.s_addr = INADDR_ANY;
215     address.sin_port = htons(PORT);
216
217     bind(server_fd, (struct sockaddr *)&address, sizeof(address));
218     listen(server_fd, 10);
219
220     printf("Chat server is running on port %d...\n", PORT);
221
222     while (1) {
223         FD_ZERO(&readfds);
224         FD_SET(server_fd, &readfds);
225         max_sd = server_fd;
226
227         for (int i = 0; i < MAX_CLIENTS; i++) {
228             int sd = clients[i].fd;
229             if (sd > 0) FD_SET(sd, &readfds);
230             if (sd > max_sd) max_sd = sd;
231         }
232
233         select(max_sd + 1, &readfds, NULL, NULL, NULL);
234
235         if (FD_ISSET(server_fd, &readfds)) {
236             new_socket = accept(server_fd, NULL, NULL);
237             for (int i = 0; i < MAX_CLIENTS; i++) {
238                 if (clients[i].fd == 0) {
239                     clients[i].fd = new_socket;
240                     break;
241                 }
242             }
243         }
244
```



```
                }
            }
        }

        for (int i = 0; i < MAX_CLIENTS; i++) {
            int sd = clients[i].fd;
            if (FD_ISSET(sd, &readfds)) {
                char buffer[BUFFER_SIZE];
                memset(buffer, 0, BUFFER_SIZE);
                int valread = read(sd, buffer, BUFFER_SIZE - 1);

                if (valread == 0) {
                    remove_client(i, 0);
                } else {
                    handle_client_message(i, buffer);
                }
            }
        }
    }
    return 0;
}
```

Kết quả:



```
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/select.h>
#include <ctype.h>

#define MAX_CLIENTS 100
#define BUFFER_SIZE 1024
#define PORT 9000

typedef struct {
    int fd;
    char nick[50];
    int is_joined;
} Client;

Client clients[MAX_CLIENTS];
char current_topic[256] = "";
char current_op[50] = "";

void send_to_client(int fd, const char *msg) {
    send(fd, msg, strlen(msg), 0);
}

void broadcast(const char *msg, int exclude_fd) {
    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (clients[i].is_joined && clients[i].fd !=
        exclude_fd && clients[i].fd != 0) {
```

```
[PASS] MSG broadcast to carol: carol-client nhan dung "MSG bob hello room

[PASS] PMSG response: alice-client nhan dung "100 OK

[PASS] PMSG to bob: bob-client nhan dung "PMSG alice secret message

[PASS] PMSG not broadcast to carol: carol-client nhan dung ""
[PASS] PMSG unknown nickname: alice-client nhan dung "202 UNKNOWN NICKNAME

[PASS] TOPIC denied for non-owner: bob-client nhan dung "203 DENIED

[PASS] OP response: alice-client nhan dung "100 OK

[PASS] OP broadcast to bob: bob-client nhan dung "OP bob

[PASS] OP broadcast to carol: carol-client nhan dung "OP bob

[PASS] TOPIC response: bob-client nhan dung "100 OK

[PASS] TOPIC broadcast to alice: alice-client nhan dung "TOPIC bob new room topic

[PASS] TOPIC broadcast to carol: carol-client nhan dung "TOPIC bob new room topic

[PASS] KICK denied for old owner: alice-client nhan dung "203 DENIED

[PASS] KICK response: bob-client nhan dung "100 OK

[PASS] KICK broadcast to alice: alice-client nhan dung "KICK carol bob

[PASS] KICK message to kicked client: carol-client nhan dung "KICK carol bob

[PASS] KICK unknown nickname: bob-client nhan dung "202 UNKNOWN NICKNAME

[PASS] QUIT response: alice-client nhan dung "100 OK

[PASS] QUIT broadcast: bob-client nhan dung "QUIT alice

Tong so test: 29, that bai: 0
tungquan@Laptop-TungQuan:~/20252_laptringmang/Code_tren_lop/20260616$
```